

SESSION 3 / 4

Industrial frameworks & multi-agents

LangGraph, CrewAI, RAG — and orchestrating multiple agents

Guillaume Hochard · Airpanel | 22–24 June 2026

Framework landscape (2025-2026)

Five references — choose based on your need for control and multi-agents.

LangGraph — explicit state graph — fine control, persistence, resumption

CrewAI — roles + tasks — agent teams, sequential or hierarchical

LlamaIndex — data-oriented agents — RAG and connection to corpora

Microsoft AutoGen — programmable multi-agent conversations

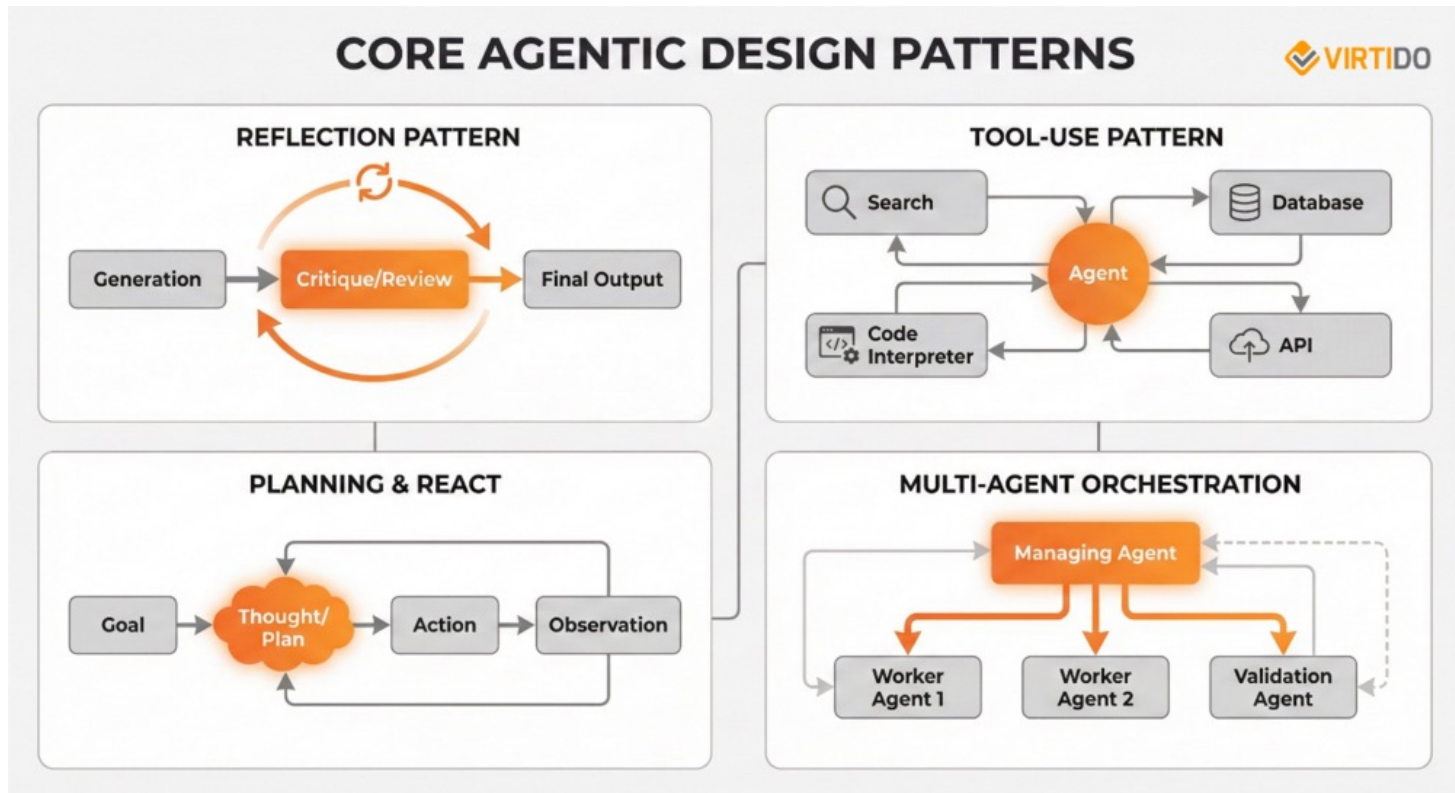
OpenAI Agents SDK — native agents, tools and handoffs, simple integration

Selection criteria

The questions to ask before adopting a framework:

- Control vs speed: explicit graph (LangGraph) or high-level abstraction (CrewAI)?
- Need for state, resumption, streaming, run persistence?
- Native multi-agent support and built-in observability?
- Maturity, community, and independence from a vendor (sovereignty)?

Agentic design patterns



Agentic design patterns

Planning

break a complex task into ordered sub-steps.

E.g. plan-and-execute

Reflection

the agent critiques and corrects its own output.

E.g. Reflexion, self-critique

Tool use

delegate computation, search, code to reliable functions.

E.g. code interpreter

Memory / RAG

retrieve relevant external knowledge.

E.g. vector store

LangGraph: the agent as a state graph

Nodes + conditional edges = a controllable, resumable loop.



LangGraph in a few lines

The framework hides the loop; you declare tools + model.

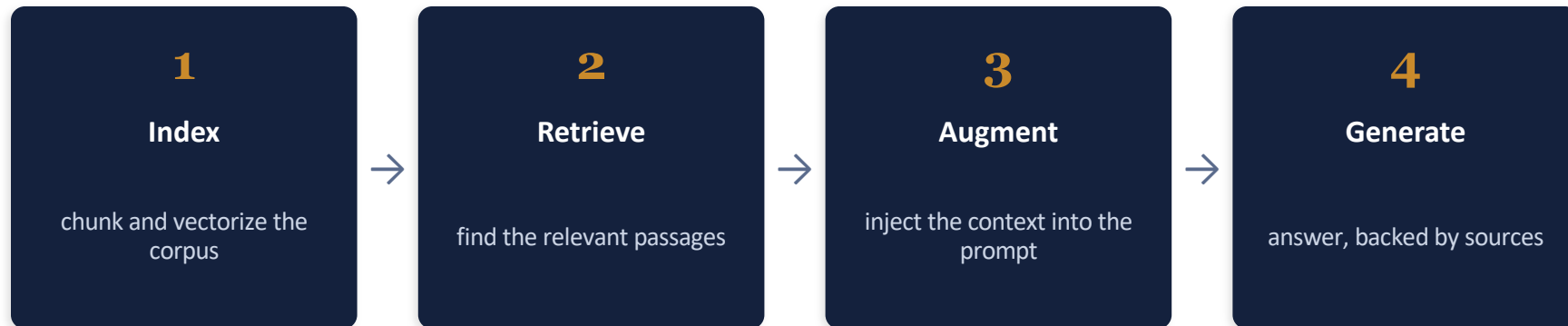
```
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

graph = create_react_agent(llm, tools=[multiply])
graph.invoke({"messages": [("user", "7 x 8 ?")]}))
```

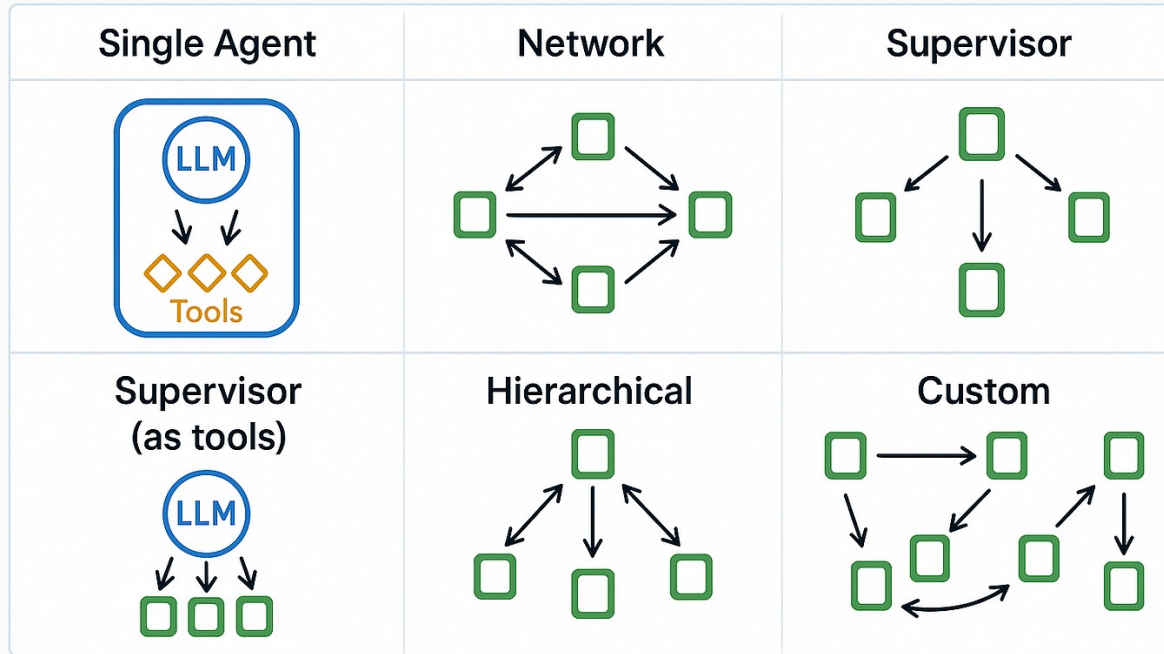
RAG in practice

Give the agent a document memory.



Multi-agent patterns

Agentic Patterns



Multi-agent patterns

Supervisor / workers

one agent distributes tasks among specialists.

E.g. manager → experts

Hierarchical

a manager delegates to sub-teams.

E.g. nested teams

Sequential pipeline

one agent's output feeds the next.

E.g. researcher → writer

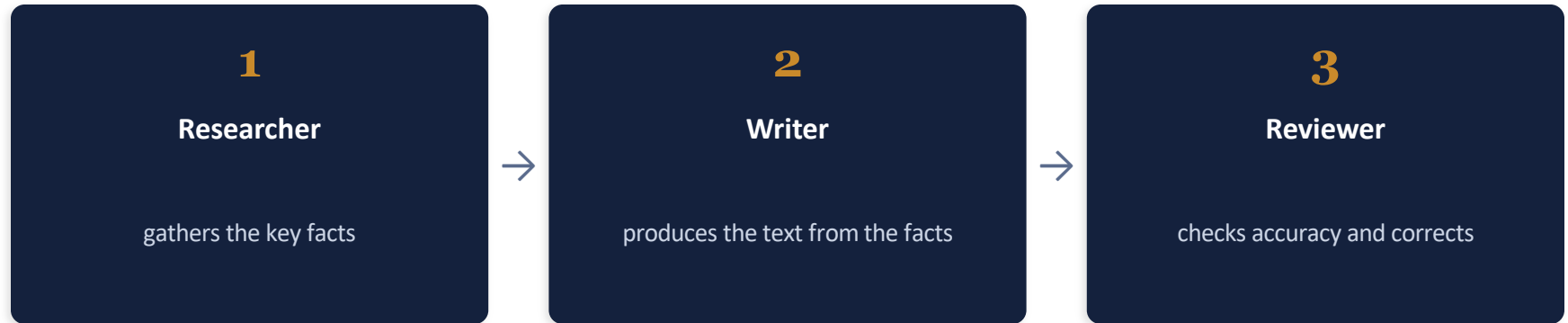
Debate / review

agents critique each other to make the answer reliable.

E.g. writer ↔ reviewer

CrewAI: a team of agents

Roles + tasks pattern implemented in Lab 3.



Persistence & human-in-the-loop

Two almost-free superpowers of LangGraph (Lab 3).

Checkpoint

saves state → memory between two runs

thread_id

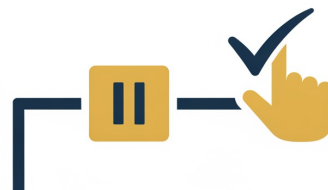
resumes the right conversation, like a session

interrupt before tools

pauses the agent BEFORE a sensitive action

Approval → resume

a human approves, then execution continues



Lab 3 — LangGraph, RAG & CrewAI

Notebook: `lab_03_frameworks_langgraph_crewai.ipynb`

- Build a ReAct agent with LangGraph (state graph + tools).
- Add a retriever and do RAG on a small corpus.
- Orchestrate a multi-agent system (researcher → writer → reviewer) with CrewAI.
- Compare cost/latency: 'from scratch' (Lab 2) vs LangGraph vs CrewAI.

Key takeaways

- Frameworks industrialize the loop coded in Lab 2.
- LangGraph for control (graph), CrewAI for agent teams.
- RAG grounds the agent in up-to-date external knowledge.
- Choose based on control, state, multi-agents and observability.
- Next step: make it reliable, secure it and deploy it.

